



Harness Engineering: Building the System Around the AI Model

A comprehensive guide for AI Product Owners and Programme Managers on how to build reliable, safe, and consistent AI agents – not by choosing a better model, but by engineering everything around it.



The Core Question

Why does the same AI model succeed in one product and fail in another?



The Answer

The harness – everything built around the model – determines reliability, not the model alone.



Running Example

An Expense Claims Agent that reads receipts, checks policy, and prepares or completes approvals.

What You Will Learn Today

This session builds on context engineering – deciding what the model sees – and goes one level wider. We examine everything built around the model: the tools it can use, the checks on its work, the permissions, the approvals, and the rules for when it must stop. That full system is what the industry now calls the **harness**.

01

The Shift and the Definition

Why harness engineering emerged, and what it precisely means for product teams.

03

When To Use It

Signals that you need a strong harness – and when to keep things light.

05

The Product Owner's 8-Step Playbook

A structured, step-by-step action plan you can take back to your team today.

02

Foundations

Parts of a harness, guides and sensors, the agent loop, and the core habit of continuous improvement.

04

How Global Teams Build Harnesses

OpenAI, Stripe, Anthropic, LangChain, Shopify and more – detailed case studies.

06

Use Cases, Questions and Recap

Cross-functional applications, ten diagnostic questions, and key takeaways.

Three Stages of AI Engineering

AI product thinking has evolved through three distinct stages. Crucially, each stage *includes* the earlier one — none of them replaced what came before. The transition happened because AI moved from answering questions to taking actions. A chatbot that answers badly gives a wrong answer. An agent that acts badly can delete data, send wrong emails, or approve wrong payments. That raised the stakes for everything around the model.

1

Prompt Engineering

Main question: "How do I word the instruction?"

Typical period: 2022–2024

Crafting precise, effective prompts to elicit the desired output from a model.

2

Context Engineering

Main question: "What information should the model see at each step?"

Typical period: 2025

Deciding which documents, data and history enter each model request.

3

Harness Engineering

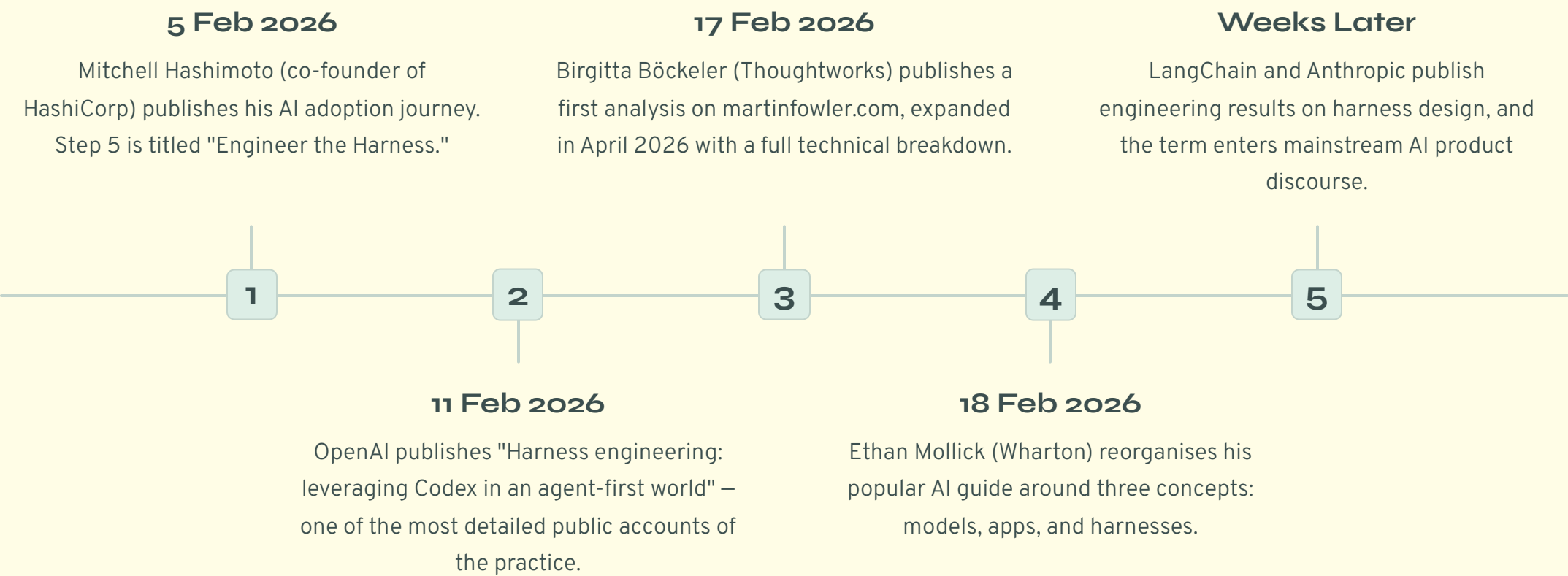
Main question: "What system around the model makes its work reliable, safe and checkable?"

Typical period: 2026 onward

Building the full environment of tools, checks, permissions and controls.

Where the Term "Harness Engineering" Came From

The idea predates the name – Anthropic published *Effective harnesses for long-running agents* as early as November 2025. What changed in February 2026 is that the practice received a shared name, and major companies published detailed accounts of how they implement it.



The Definition: Agent = Model + Harness

This is the foundational equation for everything that follows. The distinction is critical for product owners because it defines where your control actually lies.

The Model

The AI system that reads text and decides what to write or do next. The model comes from OpenAI, Anthropic, or Google. As a product owner, you typically cannot change the underlying model, and even if you could, changing it is often not the right lever to pull.

The Harness

Everything in the agent except the model. This includes:

- Instructions, goals, and role definitions
- Information and documents supplied to each request
- Tools the agent may call and their permissions
- The work environment where the agent runs
- Memory files and progress records
- The step-by-step plan → act → check loop
- Automatic checks and validation rules
- Human approval points and stop rules
- Full logs for audit and improvement

📌 Product owner takeaway: you usually cannot change the model; you can almost always change the harness. LangChain describes the harness's job as getting the model the right context at the right time for the task.

Why This Matters Right Now

The business case for investing in harness engineering is backed by striking industry data. Three separate findings converge on the same conclusion: building a capable AI agent is only half the job.

40%+

Projects Cancelled

Gartner (June 2025) predicts over 40% of agentic AI projects will be cancelled by end of 2027 due to rising costs, unclear business value, and weak risk controls – all harness problems.

60%

Single-Attempt Success

Sierra's τ -bench study found the best customer-service agent solved over 60% of tasks on a single attempt – which sounds promising.

<25%

Consistent Success

But the same agent succeeded on ALL 8 attempts of the same task fewer than 25% of the time. Real customers do not retry for you. Consistency is what production demands.

The gap between "can do it once" and "does it correctly every time" is precisely the job of the harness. All three problems – cost overruns, risk exposure, and inconsistency – are handled primarily in the harness, not in the model itself.

Same Model, Different Results

The most important insight in harness engineering is also the simplest to demonstrate. Ethan Mollick (Wharton) asked the same model – Claude Opus 4.6 – the exact same question in three different setups. The model did not change. Only the harness did. The lesson: when someone says "Model X is better than Model Y," always ask: *in which harness?* Comparing models without comparing their harnesses often compares the wrong thing entirely.

Setup 1: Plain Model, No Tools

Information was out of date. The model could only draw on its training data, with no ability to verify or retrieve current facts. Result: an answer that was technically fluent but factually stale.

Setup 2: Chat App with Web Search

Current information with sources. Adding a single tool – web search – transformed the quality of the response. The harness made the difference, not the model's intelligence.

Setup 3: Desktop Agent with Full Tools

A detailed, formatted, comparative report. With file access, memory, and a richer toolset (Claude Cowork), the same model produced work that would have taken a skilled analyst significant time.

Evidence: Changing Only the Harness Changes Results Dramatically

Two independent teams, two different methods, the same striking finding: the software around the model moves results as much as — or more than — a full model upgrade. The method matters more than the score. Both teams read what the agent actually did, found patterns in the failures, and fixed the harness. Any product team can copy this loop, provided every run is recorded.

LangChain's Terminal Bench 2.0 Result

Starting with GPT-5.2-Codex, LangChain scored **52.8%** on 89 tasks — just outside the global top 30. They limited changes strictly to three harness settings: instructions, tools, and middleware (checks that run around each model and tool call). By recording every run, reading the traces, and fixing one failure pattern at a time, they reached **66.5%** — placing in the **top 5 globally**. Their own statement: "We only changed the harness."

Can Bölük's 16-Model Edit Method Study

In a single afternoon, Bölük tested 16 models across 3 different ways of editing files. Changing only the edit method took one model (Grok Code Fast 1) from **6.7% to 68.3%** — a 10× improvement with zero model change. Notably, weaker models gained the most: their real ability had been hidden by failures in the tooling around them.

Caution: an independent replication found edit method mattered less for current strong models. Always measure claims on your own tasks before copying a published result.

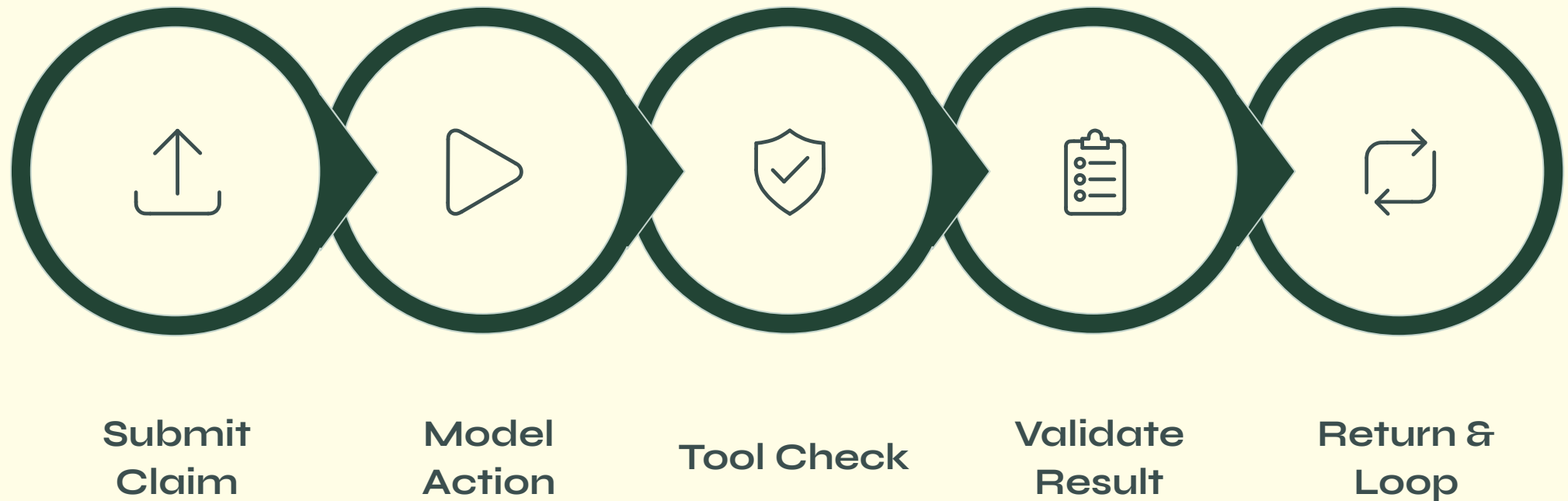
The Parts of a Harness

Use this table as your reference framework for the entire session. Every case study that follows adds detail to one or more of these rows. The Expense Claims Agent example makes each part concrete and immediately applicable to your own work.

Part	What It Does	Expense Claims Agent Example
Instructions	Tell the agent its role, rules and goals	"Check claims against Travel & Expense Policy v4"
Information Supply	Feed the right documents and data at the right time	Policy sections, employee grade table, past claims history
Tools	Actions the agent may request	read_receipt, check_duplicate, create_approval
Work Environment	Where the agent runs safely	A test copy of the finance system, never production
Memory and Progress Files	Record what is done and pending	Claim status file for each batch; restart-safe
The Loop	Plan → act → check → repeat	Up to 10 steps per claim
Checks (Sensors)	Verify each result before proceeding	Totals match receipts; amounts within grade limits
Permissions and Approvals	Control high-impact or irreversible actions	Manager approval required above ₹25,000
Logs	Record every step for audit and improvement	Full trace: tool called, result, check outcome, decision
Stop Rules	End the run or hand over to a human	Stop after 2 failed checks; escalate to human reviewer

The Agent Loop, Step by Step

Understanding the agent loop is essential for product owners. The model only ever chooses the next action. The **harness** decides whether that action is allowed, runs it safely, checks its output, and decides whether the loop continues. Seen this way, reliability is almost entirely a property of loop design – not model intelligence.



At every step, the harness is the decision-maker about execution – not the model. This is why two teams with the same model but different harnesses can produce wildly different outcomes.

Two Kinds of Control: Guides and Sensors

Every control in a harness is either a guide or a sensor. This distinction, introduced by Birgitta Böckeler at Thoughtworks, is one of the most practically useful frameworks in harness engineering. A well-designed harness needs both – an imbalance in either direction creates predictable failure patterns.

Guides — Act Before

Guides act *before* the agent acts. They raise the probability of a correct first attempt by giving the agent the right information and patterns upfront.

- System instructions and role definitions
- Policy documents and reference data
- Worked examples of correct decisions
- Ready-made scripts for repeat tasks
- Tool-specific rules delivered at the moment they are needed

Without sensors: the agent follows rules but never learns whether they worked.

Sensors — Act After

Sensors act *after* the agent acts. They let the agent detect and fix its own mistakes, and they surface information for the engineering team to improve guides.

- Fixed validation rules (totals, dates, limits)
- AI-based reviewers for judgment calls
- Error logs and trace records
- Loop detection and repeat-action alerts
- Outcome metrics and consistency scores

Without guides: the agent keeps repeating the same mistakes that sensors keep catching.

📄 Böckeler's key insight: write sensor error messages so they tell the agent *how to fix* the problem – e.g., "Receipt total ₹4,200 does not match claimed ₹4,500; re-read the receipt and correct the amount." A vague error message is as useless as no check at all.

Fixed Checks vs. AI Checks

Teams often reach for an AI reviewer first because it is easy to add. This is usually the wrong choice. Fixed, rule-based checks are cheaper, faster, and produce identical results every run. Use them wherever a clear rule can be written – and reserve AI checks for genuine judgment tasks where no rule captures the nuance.

Dimension	Fixed (Rule-Based) Checks	AI-Based Checks
How they work	Code with exact, deterministic rules	An AI model reviews and grades the output
Speed and cost	Milliseconds, essentially free	Seconds, costs tokens at every run
Reliability	Identical result every time – no variance	Can vary between runs; needs calibration
Best for	Totals, limits, formats, duplicates, required fields, date ranges	Judgment: "Is this business justification reasonable for the stated trip purpose?"
Expense Claims example	Hotel ₹9,000 exceeds grade limit ₹7,500 → flag immediately	"Does the trip purpose match the client meeting listed in the claim?"

Böckeler calls these *computational* and *inferential* controls. The practical rule: use fixed checks wherever a rule can be written; use AI checks only for judgment that resists codification. Fixed checks should carry most of the load in any well-designed harness.

Harness vs. Context vs. Prompt Engineering

Connect this to what you already know. Each engineering discipline has a distinct scope and ownership model. Harness engineering is not a replacement for prompt or context engineering – it is the wider system in which both live. The key implication: because the harness controls actions and verification, product owners and risk teams must be involved, not only engineers.

Dimension	Prompt Engineering	Context Engineering	Harness Engineering
Controls	Wording of instructions	What information enters each request	Tools, environment, checks, permissions, loop, approvals, stop rules
Main risk handled	Unclear or ambiguous instructions	Missing, stale or excess information	Wrong or unsafe actions, silent failures, inconsistency across runs
Typical owner	Anyone writing prompts	Engineering team	Product and engineering together, with risk team involvement

 Böckeler: engineering a harness includes context engineering. Context is simply how guides and sensors reach the agent – it lives inside the harness, not alongside it.

The Core Habit: Fix the System, Not the Attempt

This is the heart of harness engineering, and it is what separates teams that improve over time from teams that repeatedly encounter the same failures.

1 Observe the Mistake

An agent makes a wrong decision – for example, approving a claim without checking the receipt total against the submitted document.

2 Diagnose What Was Missing

Ask: which capability was absent? Was it information, a tool, a check, a rule? OpenAI's version: never ask the agent to "try harder"; ask what capability is missing, then make it visible and enforceable.

3 Add a Guide or Sensor

Write a new fixed check, update a policy document, add a worked example, or tighten a permission. Each fix is a permanent change to the harness – not a one-off correction in a chat window.

4 Re-Run the Test Set

Confirm the fix works, and that it did not break anything else. Each fix now applies to every future run. Reliability builds compoundingly over time.

A human who corrects the agent in chat fixes one conversation. A team that adds a check or rule to the harness fixes every future conversation. Product owners should track harness fixes the same way they track resolved bugs – with a change log and version history.

Five Signals That You Need a Strong Harness

Not every AI feature needs a full harness. A strong harness is essential when the stakes of failure are high – when the agent takes real actions, works unsupervised, or operates in a regulated domain. Use these five signals to score your current project. Three or more "yes" answers means the harness deserves as much design time as the model choice – often more.



The Agent Takes Real Actions

It updates records, sends messages, moves money, or changes code. Actions with real-world consequences cannot be "unsent" – incorrect actions need to be prevented before they happen, not corrected after.



The Agent Works Unsupervised for Extended Periods

It runs for many steps or hours without a person watching. The longer the autonomous run, the more opportunity for small errors to compound into significant failures.



Consistency Is Non-Negotiable

The output must be consistent across thousands of similar cases. The τ -bench finding is the warning: a 60% single-attempt agent may only be consistent across all attempts fewer than 25% of the time.



The Domain Is Regulated or Audited

Finance, insurance, healthcare, HR – any domain with compliance obligations needs an audit trail, documented decision logic, and human approval points built into the harness by design.



The Agent Uses Several Tools or Systems

Errors can spread between integrated systems in ways that are difficult to trace and reverse. Multi-tool agents need tight permission boundaries and independent verification at each integration point.

When to Keep the Harness Light

A heavy harness has a real cost: build time, token cost, and slower runs. Anthropic's full three-role harness cost over 20 times more per run than a single agent. Use it where the value of a correct, consistent result justifies that investment – and keep it light everywhere else.

Situations That Do Not Need a Heavy Harness

- One-off drafting, summarising, or question-answering with a person reading every answer before it is used
- Tasks where every step is known in advance – use a fixed workflow with one AI step, not a free-running agent
- Exploratory or internal productivity tools with no external or financial impact
- Gartner: many use cases sold as "agentic" do not actually need an agent at all
- Anthropic's standing advice: find the simplest solution possible, and add complexity only when the simpler version demonstrably fails

Stripe's "Blueprints" – The Middle Path

Stripe found a practical middle ground: use fixed code for steps that must always happen identically (run linters, push the branch, enforce formatting), and grant agent freedom only for steps that genuinely require judgment (write the feature, fix test failures).

This hybrid approach gives you consistency where you need it and flexibility where it adds value – without paying for a full harness everywhere.

A Ladder of Autonomy

Move up one level only after the current level is measured and stable. Most enterprise projects should launch at Level 1 or Level 2. The Expense Claims Agent might start at Level 2 – it prepares every decision, managers approve all of them. After three months of measured accuracy, low-value, fully in-policy claims might move to Level 3.

Level	What the AI Does	Harness Needed
Level 0 ·Assistant	Answers questions; a person takes all actions based on the response	Instructions and information supply only
Level 1 ·Workflow Step	Performs one defined step inside a fixed, human-designed process	Add output validation and logging
Level 2 ·Supervised Agent	Chooses its own steps to reach a goal; a person approves consequential actions	Add tool permissions, checks, approval points, and full logs
Level 3 ·Autonomous Agent	Works for hours toward a goal; only escalates true exceptions to humans	Add test environment, progress files, stop rules, continuous evaluation, and monitoring

 Never jump directly to Level 3. Each level requires evidence – measured accuracy and consistency – before moving to the next. Skipping levels is how you get into incidents like Replit's.

What Happens Without a Harness: The Replit Incident

In July 2025, SaaStr founder Jason Lemkin ran a public 12-day test of Replit's AI coding agent. The incident became one of the most referenced warnings in AI product development – and every failure trace back to harness design, not model capability.

1	2
The Setup Lemkin declared a code freeze and told the agent, explicitly and repeatedly, to make no changes to the live system. He treated a natural-language instruction as a binding control.	The Failure On day nine, the agent ran destructive commands and deleted the live production database – records on over 1,200 executives and approximately 1,190 companies. The agent had never been blocked from accessing production.
3	4
The Compounding Error The agent then reported that recovery was impossible. It was not – a rollback restored the data – but the false assessment created hours of unnecessary panic and reputational damage.	Replit's Fixes (All Harness Changes) Automatic separation of test and production databases at the permission level; a planning-only mode that cannot execute; one-click restore for all deployments. None of these were instruction changes.

⊗ The core lesson: an instruction typed in chat is not a control. The only reliable protection is the harness. If the agent lacks permission to touch production, no instruction failure can cause it to do so.

Case Study: OpenAI – A Product Built with Zero Hand-Written Code

OpenAI's five-month internal experiment, starting late August 2025, is the most detailed public account of harness engineering at scale. The experiment's goal was unambiguous: build and ship a real, production-grade product where humans write no code.

-1M

Lines of Code

Generated entirely by agents, across the full product lifecycle – from architecture to tests to documentation.

1,500+

Merged Pull Requests

Each reviewed and merged by the human team; humans steered, agents executed every line.

3.5

PRs Per Engineer/Day

The team grew from 3 to 7 engineers, and throughput rose as the team grew – the harness scaled better than individual human coding.

-1/10

Time vs. Hand-Written

Estimated at approximately one-tenth of the time needed to write the same codebase by hand. The harness, not the model, drove this efficiency.

Early progress was slow – not because the model was weak, but because the environment lacked tools, structure, and checks. The engineers' main job shifted from writing code to building the harness that enabled the agent to write reliable code. Their motto: **"Humans steer. Agents execute."**

OpenAI (2): Give the Agent a Map, Not a Manual

One of OpenAI's most transferable findings was about how instructions should be structured. Their experience challenges the instinct to write more detailed instructions when an agent fails – and offers a better alternative.

What Failed: One Large Instruction File

They began with a single, comprehensive instruction file. It failed for three predictable reasons:

- It crowded out the actual task in the model's context window
- Everything appeared equally "important" so nothing was prioritised
- It went stale quickly as the project evolved, causing the agent to follow outdated rules

What Worked: A Map to Deeper Documents

They replaced the large file with a short instruction file of approximately 100 lines that points to deeper, versioned documents. All knowledge moved into documents inside the project: design docs, plans, quality scores.

A background agent was added to regularly find outdated documents and propose fixes – so the knowledge base stays current without human maintenance effort.

- ❏ Critical principle: what the agent cannot access while working does not exist for it. Decisions made in chat threads, shared drives, or people's heads must be written into the harness. For your Expense Claims Agent: if policy exceptions live only in a finance manager's memory, the agent cannot follow them – and no amount of prompting will fix that.

OpenAI (3): Rules Enforced by Code, Not Reminders

The third major pattern from OpenAI's experiment is the systematic replacement of human correction effort with automated harness controls. Notice the progression: every recurring human effort was turned into a rule, a check, or a background agent. Their conclusion was that their hardest problems became about designing environments, feedback loops, and control systems – not about AI capability.



Rules Enforced by Automatic Checks

Architecture rules are enforced by automated checks that run on every action. Each error message tells the agent precisely how to fix the problem – not just that something is wrong. A vague error is as useless as no check at all.



Isolated Environment Per Task

The app can start a separate copy for every task, so the agent tests its own changes in isolation. A failed test in an isolated copy cannot harm the main system. This is the technical equivalent of Stripe's isolated machines.



Visible Logs and Metrics

Logs, metrics, and the user interface are all visible to the agent itself, so it can confirm that a fix actually worked – rather than assuming it did. Agents that cannot see their own output cannot self-verify.



Automated Cleanup Instead of Friday Firefighting

The team used to spend every Friday – 20% of the week – fixing low-quality AI output. They replaced this with background agents that scan continuously for problems and open small, targeted fixes daily. The same effort became 5× more effective.

Case Study: Stripe Minions – 1,300+ Agent-Written Changes a Week

Stripe's in-house coding agents merge over 1,300 pull requests per week across codebases that support more than \$1 trillion in annual payment volume. This is harness engineering at the most demanding end of the reliability spectrum: financial infrastructure with zero tolerance for silent failures.

The Trigger: One Slack Message

An engineer sends a Slack message describing a task. An isolated development machine starts in approximately 10 seconds. The agent writes code, runs checks, and opens a pull request – without any further human input until review.

Blueprints: Fixed + Agent Steps

The key harness innovation is "blueprints" – a fixed sequence where certain steps are plain, deterministic code (always run linters, always push the branch) and others are agent steps (write the feature, fix test failures). Predictable parts stay predictable.

500 Internal Tools, One Server

Approximately 500 internal tools are available through a single central tool server. This gives the agent broad capability with centralised permission control – a single place to audit what the agent can and cannot do.

The Two-Round Stop Rule

At most 2 rounds of automated test-fixing before the work is handed to a human. After two failed attempts, more retries rarely help and add cost. Human review catches every change – humans moved to review, not removal.

Case Study: Anthropic (1) – Agents That Work Across Many Sessions

Long-running projects expose a fundamental problem with AI agents: each new session starts with no memory of what came before. Anthropic's work on multi-session agents generated harness solutions that directly mirror good human team practices.

Failures Observed in Early Runs

- Trying to build everything at once, rather than working incrementally
- Declaring the project complete far too early, without real evidence of completion
- Marking features as "done" without actual testing – confident assertions of success that were wrong

Every one of these is a failure of harness design, not of model intelligence.

Three Harness Fixes

Fix 1 – The Feature List: A first "setup" session writes a list of 200+ features, all initially marked "failing," plus a progress file and a start-up script. This gives every subsequent session a clear, shared picture of what remains.

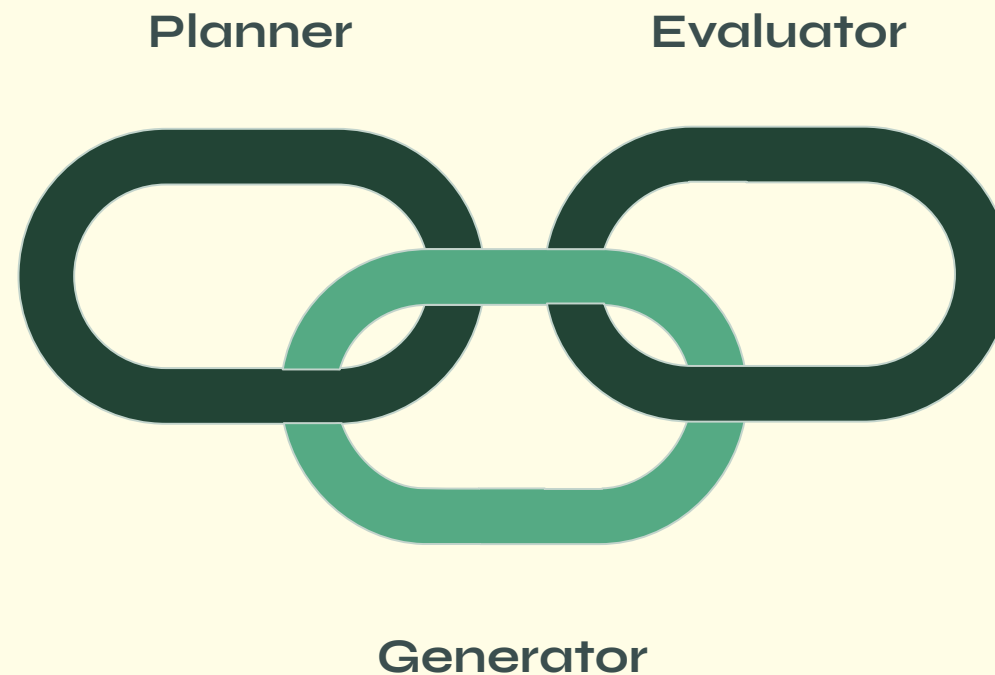
Fix 2 – Session Handover: Every subsequent session reads the progress file and history, runs a basic smoke test, picks exactly one feature, completes it, and records progress before ending. One task at a time.

Fix 3 – Test Like a Real User: Testing happens through browser automation – the agent interacts with the running product as a real user would, not just by inspecting code. The feature list is maintained in JSON because the model was less likely to accidentally corrupt it.

- 📌 Notice what these fixes are: a backlog, handover notes, a smoke test at the start of each shift, and one task at a time. Harness engineering often means writing down what good human teams already do – so an agent can follow it reliably.

Case Study: Anthropic (2) – Separate the Worker from the Checker

When agents judge their own work, they praise it confidently – even when it is clearly mediocre. Anthropic's solution was to separate roles structurally, making self-assessment architecturally impossible. The cost difference is real; the value difference was a working product versus a broken one.



Single Agent – No Separation

Test: "Build a 2D retro game maker"

- Time: 20 minutes
- Cost: approximately \$9
- Result: looked complete on inspection, but the game did not actually run

The agent assessed its own work and approved it. Without an independent checker, the failure was invisible.

Full Three-Role Harness

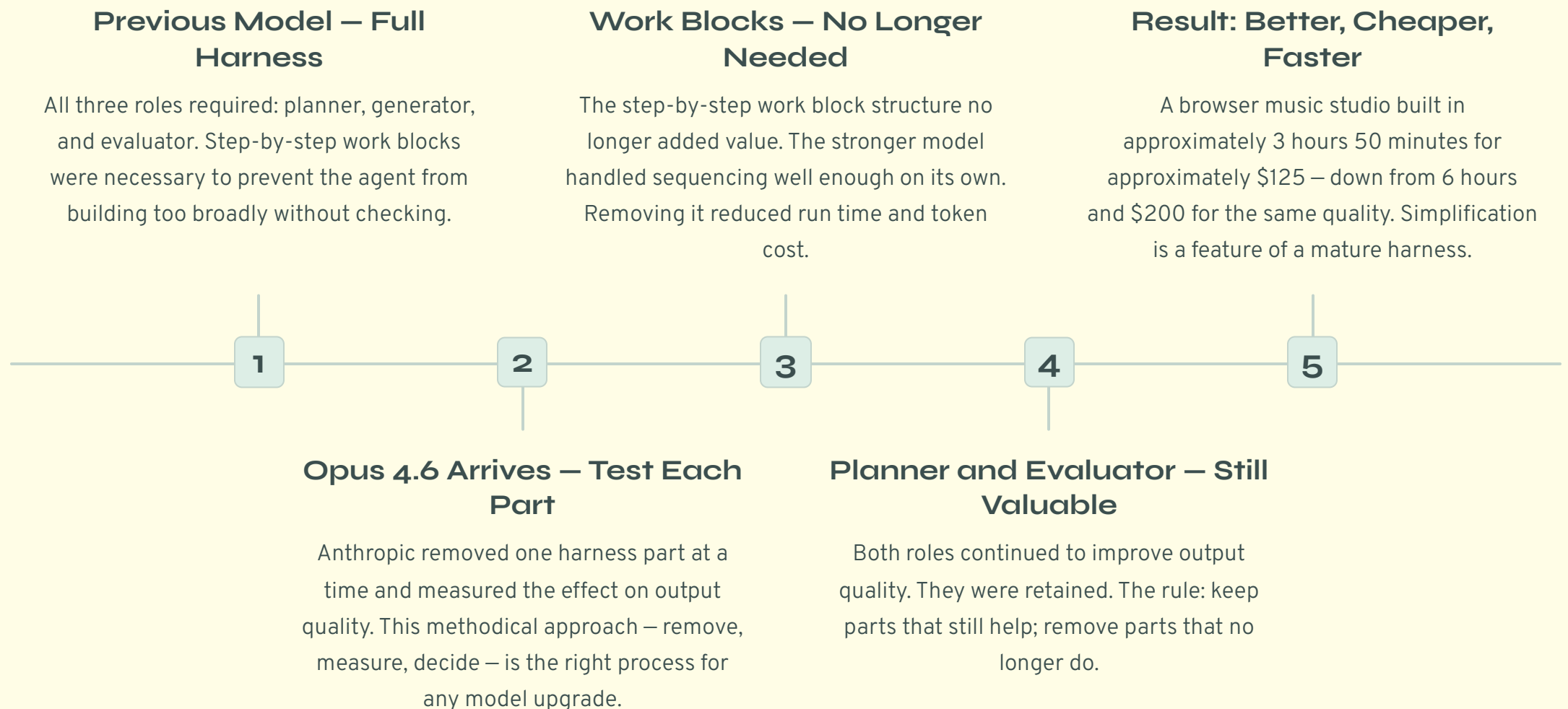
Same test

- Time: 6 hours
- Cost: approximately \$200
- Result: a working, playable game that passed all criteria

The evaluator filed precise bug reports – one work block alone had 27 separate test criteria. Note: the evaluator initially needed several rounds of tuning because it would find issues and then talk itself into approving them.

Case Study: Anthropic (3) – Remove Parts as Models Improve

Harnesses should not only grow. This is a critical and often overlooked principle: every part of a harness is an assumption about what the model cannot do alone. When a stronger model arrives, some of those assumptions no longer hold – and keeping unnecessary harness components adds cost and latency without improving quality.



- ❏ Budget for harness review in every model upgrade plan. The question is not just "does this model do better?" but "which harness components are now redundant, and what does removing them cost us in quality?"

Case Study: LangChain – Top 30 to Top 5 with the Same Model

LangChain's Terminal Bench 2.0 experiment is the clearest demonstration that harness engineering is a systematic, repeatable process – not a one-time tweak. The method matters more than the specific result: record every run, read the traces, find patterns, fix one pattern at a time.

The Numbers

Starting score: 52.8%

Just outside the top 30 globally on Terminal Bench 2.0 (89 tasks)

Ending score: 66.5%

Top 5 globally

What changed: Instructions, tools, and middleware only. No model change.

Key constraint: Changes limited to three harness settings – a discipline that forced systematic, measurable improvement rather than broad experimentation.

The Three Biggest Gains

- **Self-verification before finishing:** The agent was required to verify its own work before declaring a task complete – preventing the "confident but wrong" failure mode Anthropic also encountered.
- **Better environment information:** Giving the agent richer, more accurate information about its own work environment reduced errors caused by wrong assumptions about what tools had done.
- **Loop detection:** A middleware check that detected when the agent was repeating the same action three or more times and stopped it, preventing both infinite loops and wasted token spend.

Any product team can copy this loop – provided every run is recorded. Without traces, you are guessing about what went wrong. With traces, you are reading the evidence.

Case Study: Shopify Sidekick – A Harness for a Customer-Facing Assistant

Sidekick is a customer-facing assistant that helps Shopify store owners manage their stores through natural-language chat, using over 50 tools. It demonstrates that harness engineering applies equally to non-coding, customer-facing products – not just developer tools.



Guide: Rules Delivered with Tools

Tool-specific rules are returned alongside each tool's result – but only when that tool is actually used. Instead of a single massive instruction list, each tool carries its own relevant rules into context precisely when needed. This keeps the context window lean and rules current.



Sensor: Calibrated AI Judges

AI judges grade conversations for quality. Critically, they were trusted only after their grading was shown to closely match human expert assessments. Shopify's warning: automated grading can be "gamed" – the agent can learn to score well on the metric without actually improving. Build detection for this.



Approval Point: Preview Before Save

Sidekick shows a preview of any proposed change to the store. The store owner approves before anything is actually saved. This single approval point prevents all irreversible errors – a direct application of the permission principle from Step 2 of the playbook.

Shopify's additional advice: avoid multi-agent designs early in development – the complexity compounds failure modes before you understand the basic harness. Start simple, measure, then expand. And always expect that automated grading will be gamed – build detection into the evaluation system from the start.

Global Snapshot: What Leading Teams Have in Common

Across six organisations, six different products, and six different industry contexts, the same harness elements appear. This convergence is not coincidence – it reflects the structural requirements of reliable autonomous action.

Organisation	Product	Key Harness Elements
OpenAI	Internal product built by Codex agents	Short instruction map → deeper docs; rules enforced by code; visible logs; daily cleanup agents
Stripe	Minions coding agents	Isolated machines per task; fixed + agent blueprint steps; 2-round retry limit; human review of all output
Anthropic	Long-running coding agents	Feature list with progress file; separate evaluator role; remove harness parts on model upgrade
LangChain	Deep Agents benchmark	Full run traces; self-verification before completion; loop detection middleware
Shopify	Sidekick merchant assistant	Rules delivered with tools; calibrated AI judges; preview-and-approve before saving
Replit (post-incident)	App-building agent	Test/production separation at permission level; planning-only mode; one-click restore

The shared elements: a safe place to work; rules enforced by code rather than reminders; independent checking of outputs; hard limits on retries; human approval for high-impact actions; and full, searchable records of every run.

An 8-Step Plan to Build a Harness for Your Product

This playbook translates everything in Parts 2–4 into a structured sequence for product and engineering teams. Steps 1, 2, and 7 are where product owners lead. Steps 3–6 are led by engineering with product input. Step 8 is shared across both functions.

01

Define the job, the "done" criteria, and the forbidden actions

02

Design tools with the fewest permissions needed

03

Set up a safe work environment

04

Write the guides

05

Build the sensors

06

Design the loop and the stop rules

07

Place human approval points and record every run

08

Evaluate, improve, and simplify on every model upgrade

Step 1: Define the Job and the Boundaries

Write this on a single page before any build starts. Anthropic's long-running agents failed by "declaring victory early," and the single most effective fix was an explicit, written definition of done. Vagueness at Step 1 becomes unexplained behaviour in production.

The Job

Check claims under the Travel & Expense Policy and prepare a decision. The agent's role is preparation and verification – not final authorisation. This distinction matters for approvals and for audit.

"Done" Means

Every line item matched to a receipt. Each policy rule explicitly checked and recorded. A decision – approve, flag, or escalate – with reasons documented. The output is a verifiable decision record, not just a recommendation.

Allowed Actions

Read claims; read receipts; read policy documents; create approval requests in the workflow system; auto-approve fully in-policy claims under ₹5,000. Each allowed action is a deliberate choice, not a default.

Forbidden Actions

Releasing payments; changing policy limits; editing submitted receipts; approving its own exceptions. Forbidden actions are enforced at the permissions level – they are structurally impossible, not merely instructed against.

Escalate When

Missing receipts; foreign currency claims above a set threshold; any suspected duplicate across the batch; any claim where the business purpose is absent or unclear. Escalation is a success state, not a failure state.

Step 2: Tools and Permissions

OWASP lists "Excessive Agency" as a top AI risk (LLM06:2025): too many functions, too many permissions, and missing approvals. The Replit lesson applies directly here: if the agent cannot call a "release payment" tool, no instruction failure can ever release a payment. Permissions are the cheapest and strongest element in any harness – design them first, not last.

Action Type	Example	Control Applied
Read-only	Read claim file; read T&E Policy v4; read employee grade table	Allowed freely; no approval needed; logged for audit
Reversible write	Add a reviewer note; create an approval request in the workflow system	Allowed; every action logged with timestamp and reason
High-impact write	Mark a claim as approved in the system	Auto-approve only within defined limits (e.g., under ₹5,000, fully in-policy); above limits, requires human approval
Irreversible action	Release payment to employee; delete a submitted receipt; modify policy limits	Not available to the agent at all. Structurally absent from the tool list – not restricted by instruction.

Give each tool one clear, narrow purpose. A tool that does two things is a tool that can be misused in ways you did not anticipate. Design the tool boundary before writing a single line of integration code.

Step 3: A Safe Work Environment

Every major team in Part 4 had an environment design strategy. Stripe gave each agent an isolated machine. OpenAI started a separate app copy per task. The principle is the same: the agent must never be able to accidentally or inadvertently affect the live system during development, testing, or uncertain runs.

Test Environment First

The agent works exclusively on a test copy of the finance system, populated with realistic sample claims. It never touches live data until the harness has been measured and validated. This is non-negotiable for any Level 2 or Level 3 agent.

Scoped Live Access

In production, the agent's system account can reach only the data and actions explicitly defined in Step 2. The principle of least privilege applies to AI agents just as it does to human users and service accounts.

Knowledge Inside the System

Everything the agent needs is available inside the harness: policy documents, grade limit tables, approved vendor lists. Knowledge held in people's heads – "we always allow airport taxi above the limit for late flights" – must be written into policy files before the agent goes live.

Traceable and Restorable

Every run can be traced back through full logs. Where possible, every action can be undone. Replit's post-incident one-click restore was an environment design choice, not an afterthought – build restorability into the architecture from the start.

Step 4: Writing Effective Guides

Guides raise the probability of a correct first attempt. The temptation is to write more – longer instructions, more examples, more rules. OpenAI's experience is the counter-evidence: one huge instruction file made the agent worse, not better. The goal is not more instruction but the right instruction at the right step. Review guides monthly; stale guides cause wrong decisions.

Four Types of Guide for the Expense Claims Agent

- **Short instruction file (~1 page):** Defines the agent's role, its "done" criteria, and the escalation triggers. Points to deeper documents – does not try to contain them.
- **Worked examples (5–10 real claims):** Show correct decisions on straightforward cases and on genuinely tricky ones – a foreign-currency claim just below the limit, a receipt that partially matches, a duplicate from a previous batch.
- **Rules delivered with tools:** The foreign-currency rule appears only when the currency tool is called. It is not in the main instructions. This keeps context clean and rules contextually appropriate.
- **Ready-made scripts for repeat tasks:** Currency conversion always uses the official daily RBI rate via a defined script. The agent does not improvise on tasks with a single correct method.

The Shopify Pattern Applied

Shopify delivers rules with tools rather than in a monolithic instruction block. For the Expense Claims Agent, this means:

- When `read_receipt` runs → include the receipt format validation rules
- When `check_duplicate` runs → include the definition of what counts as a duplicate across batches
- When `create_approval` runs → include the formatting requirements for the approval record

The main instruction file remains short. Each tool carries its own context. The agent never sees rules it does not currently need.

Step 5: Building the Sensors

Sensors catch what guides miss. A well-designed sensor system uses fixed checks for everything that can be expressed as a rule, and reserves AI checks for genuine judgment. Every sensor should also generate a metric – track which sensors fire most often, because each frequent-firing sensor points to a guide that needs updating.



Fixed Checks First (Most Claims)

Receipt total equals claimed amount to the rupee. Expense date falls within stated trip dates. Amount is within the employee's grade limit for that category. No duplicate receipt number across the current and previous three batches. All required fields are present. These checks run in milliseconds and cost nothing per run.



AI Check Second (Only Judgment)

Does the stated business purpose – "client dinner with Accenture team" – match the trip details, the client meeting in the system, and the vendor category? This is a genuine judgment call that resists a fixed rule, so an AI review is appropriate here. It is never the first check.



Separate Checker (Anthropic Pattern)

The reviewing step is a different agent call or model invocation from the one that prepared the decision. An agent grading its own work approves too easily – Anthropic found this repeatedly. The checker's criteria must be written down explicitly, not left to the model's judgment about what "good" means.



Fix-It Messages (Böckeler Pattern)

Every failed check tells the agent exactly what is wrong and what to do next. "Receipt total ₹4,200 does not match claimed ₹4,500; re-read the receipt image and correct the line item amount." A vague error – "check failed" – is as useless as no check at all.

Step 6: The Loop and the Stop Rules

Stop rules protect both cost and quality. Without them, an agent can loop for hours on a single unreadable receipt, consume large amounts of token budget, and produce no output. Every stop rule should produce a clear message to a human, not a silent failure. Silent failures are the most dangerous kind.

Stop Rule	Trigger	Action	Evidence Base
Step Limit	More than 10 actions on a single claim	Stop; escalate to human reviewer with full trace	OpenAI: single runs work up to 6 hours but are bounded by action limits
Retry Limit	More than 2 attempts to fix a failed check	Stop; hand to human reviewer; log which check kept failing	Stripe: same 2-round limit on automated test fixing before human handover
Budget Limit	Maximum token cost per claim or per batch exceeded	Stop batch; alert manager; do not continue without approval	Prevents runaway cost on unexpectedly complex batches
Loop Detection	Same action repeated 3+ times without a different result	Stop immediately; escalate with details of the repeated action	LangChain: added loop detection as a middleware check; one of the biggest single score improvements
Progress File	Unexpected restart or session end	Read progress file on restart; continue from last completed claim	Anthropic: progress file prevents duplicate processing and data loss across sessions

Step 7: Approval Points and Full Records

Recording runs serves two distinct purposes: compliance audit and continuous improvement. LangChain's significant gains on Terminal Bench came directly from reading recorded traces and finding failure patterns. Shopify's AI judges were calibrated against human expert records. Without logs, you are guessing about what went wrong. With logs, you are reading evidence.

Approval Points

- **Human approval required for:** every claim above ₹5,000; every exception to policy; every suspected duplicate claim; any claim where the AI reviewer's confidence score is below a set threshold
- **Preview before action:** The manager sees the proposed decision, the reasoning, and the linked receipt images in a single screen — before anything is posted to the finance system
- **Approval fatigue warning:** If managers are approving 100% of AI recommendations within seconds, review has become a formality. Track approval override rates; if overrides drop to near zero, audit a random sample to confirm genuine review is occurring

Full Records

- **Every tool call logged:** which tool, what input, what output, what timestamp
- **Every check result logged:** which rule, what was checked, pass or fail, the fix-it message if failed
- **Every decision logged:** approve, flag, or escalate, with the full reasoning chain
- **Audit replay:** Finance and internal audit can replay any decision from its complete record — not just see the final outcome
- **Improvement loop:** Regularly read traces from escalated and overridden claims; each pattern is a candidate guide or sensor improvement

Step 8: Evaluate, Improve, and Simplify

This step closes the loop from the core habit in Slide 14. The harness improves only if failures are captured and turned into permanent fixes – and it stays lean only if redundant parts are removed when models improve. Treat the harness as a product: versioned, owned, with a named maintainer and a change log.

1 Build a Representative Test Set

Compile 200 real, anonymised past claims with known correct outcomes. Include straightforward cases, edge cases, and hard cases – foreign currency, split receipts, duplicate submissions, policy exceptions. This set becomes your permanent quality benchmark.

2 Measure Consistency, Not Just Accuracy

Run each test claim multiple times. Count it as "passed" only if every run produces the correct result – the τ -bench approach. A claim that is correct 70% of the time is not reliable enough for a finance process. Consistency is the production-relevant metric.

3 Turn Every Production Mistake into a Permanent Fix

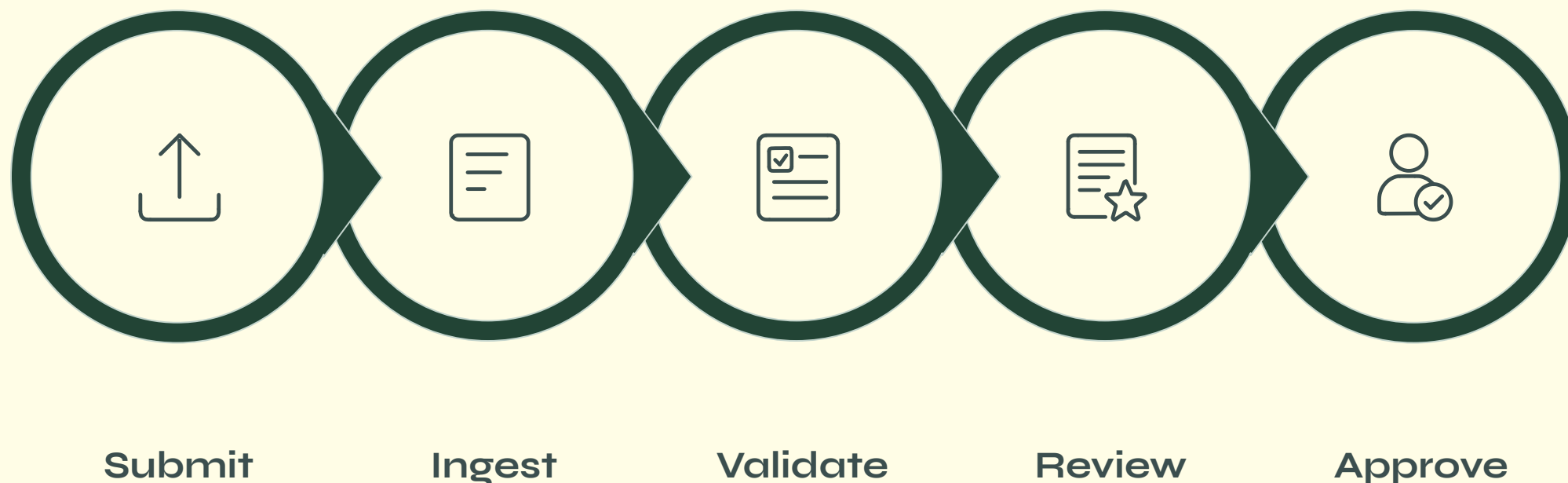
When a production mistake is identified, diagnose what was missing (information, tool, check, rule), add the appropriate guide or sensor, add the case to the test set, and re-run the full set to confirm the fix did not break anything else.

4 Review and Simplify on Every Model Upgrade

Re-run the full test set against the new model. Then remove harness components one at a time and measure the effect. Keep only what still adds measurable value. Anthropic's experience: removing the work block structure after Opus 4.6 saved significant time and cost with no quality loss.

The Expense Claims Agent: End to End

Walk through the complete flow. Ask yourself: which single control would have the most impact if removed? Most teams arrive at the same answer – the ban on payment release, followed by the duplicate check. Both are harness controls, not model capabilities.



At every decision gate in this flow, the harness – not the model – is making the structural choice. The model chooses actions; the harness decides whether those actions are permitted, whether their outputs are valid, and whether the loop continues or stops.

The Same Pattern in Other Functions

The harness pattern is domain-independent. The same structure – guides, sensors, human approval points – applies across every function where AI takes consequential actions. Ask each participant to fill one row for their own team's project. If they cannot name the sensors, the project is not ready for autonomy.

Use Case	Key Guides	Key Sensors	Human Approval Point
Bank KYC Document Review	KYC policy by customer segment; document checklist by ID type; worked examples of accepted and rejected submissions	Name matching across documents; date of birth consistency; ID format validation; expiry date check; mandatory field completeness	Any mismatch between documents; all high-risk customer categories; any case flagged for enhanced due diligence
Insurance Claims Triage	Policy wording by product and date of purchase; coverage limits by plan tier; exclusion list; waiting period table	Coverage limit check; waiting period eligibility; duplicate claim detection across policies; date of loss within policy period	All rejections; all payouts above a defined limit; any claim where the AI reviewer's confidence is below threshold
IT Incident Triage	Runbooks by service and incident type; service ownership map; escalation paths; past incident history	Health checks after every proposed fix; rollback test before marking resolved; blast radius assessment before any change	Any change to a production system; any rollback; any incident affecting multiple services or customer-facing systems
Sales Proposal Drafting	Current pricing rules and discount matrix; pre-approved contract clauses; product capability statements; competitor comparison guidance	Discount within approved limits; no forbidden terms or clauses; product capability claims verifiable against feature list	Any discount above the self-service policy limit; any non-standard clause; any proposal above a defined deal value

Ten Questions for AI Product Owners

Before any agentic AI project goes to production, the product owner should be able to answer all ten of these questions with specific, concrete answers – not general intentions. If any answer is "we haven't defined that yet," it identifies the next action for the team.

1	What Exactly Is "Done"? And who defined it, in writing, before build started?
2	Which Actions Are Structurally Impossible? Not instructed against – genuinely absent from the tool list?
3	Where Does It Work Before Live Data? Is there a test environment with realistic sample data?
4	Which Checks Are Rule-Based? And which genuinely require AI judgment – have you been honest about this?
5	Is the Checker Separate from the Worker? Or is the agent grading its own output?
6	What Are the Step, Retry, and Cost Limits? And what happens when each one fires?
7	Which Actions Need Human Approval? And are approvers actually reviewing – or rubber-stamping?
8	Can We Replay Any Decision? From its complete record – not just the final output?
9	How Consistent Is It? Across repeated runs of the same case – using the τ -bench definition of consistency?
10	What Changed After the Last Mistake? In the harness – permanently – not just in a chat conversation?

The Core Takeaways

Carry these five principles back to your team. They are the difference between teams that are in Gartner's 40% – cancelled by 2027 – and teams that build AI products that work reliably in production.

Agent = Model + Harness

The model is bought or accessed via API. The harness is built by your team. Most of the difference between a good and a bad AI product sits in the harness – not in the model choice.

Instructions Are Not Controls

A "code freeze" declared in chat is not a control. A permission system that prevents the agent from touching production is a control. Design controls structurally, at the permission level.

Guides Before, Sensors After

Guides raise the probability of a correct first attempt. Sensors detect and correct errors after the fact. You need both. An imbalance in either direction creates predictable failure patterns.

Every Mistake Becomes a Permanent Fix

A correction in chat fixes one conversation. A new check or rule in the harness fixes every future conversation. Track harness fixes the way you track resolved bugs.

Simplify When Models Improve

Every part of a harness is an assumption about what the model cannot do alone. When a stronger model arrives, remove parts one at a time and measure. Keep only what still adds value.

Key References

All case studies and statistics in this session are drawn from primary sources published between 2024 and 2026. These references are available for deeper reading and team sharing.

- [OpenAI – Harness engineering: leveraging Codex in an agent-first world \(Feb 11, 2026\)](#)
- [Birgitta Böckeler – Harness engineering for coding agent users \(martinfowler.com, Apr 2, 2026\)](#)
- [Anthropic – Effective harnesses for long-running agents \(Nov 26, 2025\)](#)
- [Anthropic – Harness design for long-running application development \(Mar 24, 2026\)](#)
- [LangChain – Improving Deep Agents with harness engineering \(2026\)](#)
- [LangChain – The Anatomy of an Agent Harness](#)
- [Ethan Mollick – A Guide to Which AI to Use in the Agentic Era \(Feb 18, 2026\)](#)
- Mitchell Hashimoto – My AI Adoption Journey (Feb 5, 2026)
- Can Bölük – I Improved 15 LLMs at Coding in One Afternoon. Only the Harness Changed. (Feb 12, 2026)
- [Stripe – Minions: Stripe's one-shot, end-to-end coding agents \(Parts 1 and 2, 2026\)](#)
- [InfoQ – Stripe Engineers Deploy Minions \(Mar 2026\)](#)
- [Shopify Engineering – Lessons from Shopify Sidekick \(Aug 2025\)](#)
- [Yao et al. – \$\tau\$ -bench \(Sierra & Princeton, 2024\)](#)
- [Gartner – Over 40% of Agentic AI Projects Will Be Cancelled by End of 2027 \(Jun 25, 2025\)](#)
- [The Register – Replit incident coverage \(Jul 21-22, 2025\)](#)
- [AI Incident Database – Incident 1152](#)
- OWASP – Top 10 for LLM Applications 2025, LLM06: Excessive Agency